PyCC.id: A package for hypothesis-driven equation discovery with structural identifiability

February 10, 2026

Summary

Data-driven equation discovery is fundamentally an inverse problem that seeks to infer the governing differential equations of a system directly from time-series measurements. A known issue is the ill-conditioned nature of the inverse problem, which frequently produces multiple mathematical models that fit the data similarly well. One path to address this issue is by incorporating known hypotheses and constraints into the training phase beforehand. While this approach effectively reduces the search space, it still results in multiple candidate models, forcing practitioners to rely on post-hoc manual filtering based on their own domain expertise. A recent approach incorporates structural 'skeletons' inspired by characteristic curves (CCs), defining a hypothesis-driven methodology. In this methodology, practitioners define a skeleton, which is associated with a family of ordinary differential equations (ODEs), and then add their hypotheses and priors based on their domain knowledge to refine the obtained model iteratively. An important advantage of this approach is that some skeletons have demonstrable structural identifiability properties, which are useful for checking whether the skeleton is correct or should be discarded. Furthermore, this formalism enables the use of multiple equation discovery paradigms due to its modularity (such as neural networks, symbolic regression, and sparse regression).
In this work, we present the Python library `PyCC`, which condenses these efforts into a flexible tool that allows researchers and engineers to seamlessly define their skeletons and hypotheses to discover ODEs from time-dependent data.

Statement of Need

Data-driven equation discovery is fundamentally an inverse problem that seeks to infer the governing differential equations of a system from time-series measurements. Historically, this topic is rooted in the field of system identification within control theory. However, modern equation discovery methods have introduced novel developments that have enabled applications across a wide range of disciplines in science, medicine, and engineering. During the last two decades, this interdisciplinary effort has contributed significantly to the emerging field of scientific machine learning [1].

Despite these advances, the inverse problem remains typically ill-conditioned [2], as many different mathematical expressions can fit a finite dataset with similar accuracy. Consequently, the algorithms often provide a vast landscape of candidate models. Although these models are valid in terms of data fitting, they often vary significantly in their structural form, leading to physically inconsistent representations. Therefore, practitioners must often rely on post-hoc analysis to manually discard models that do not align with the physics based on their domain expertise. This leaves the final selection of a model somewhat arbitrary.

To address this difficulty, an emerging method incorporates the concepts of structural 'skeletons' and CCs [3, 4, 5, 6]. An structural skeleton corresponds to a given classification of families of ODEs. The choice of the structural skeleton directly affects identifiability: while certain skeletons possess structural identifiability, others can result in non-identifiable or ambiguous representations. When a skeleton is shown to be theoretically identifiable in phase space [6], it provides a formal framework to determine whether a proposed equation is consistent with the data or should be reformulated.

This work introduces `PyCC.id` (also referred to as `PyCC`), an open-source library that implements this framework through a hypothesis-driven methodology. This hypothesis-driven workflow requires the practitioner to propose a specific structural skeleton based on physical intuition or domain knowledge. `PyCC` then allows the user to integrate prior information (such as explicit physical symmetries and constraints) directly into the functions defined within these skeletons. By providing a clear pathway to validate or eliminate these hypothesized structures at the algorithmic level, the library contributes to addressing the ill-conditioned nature of the inverse problem. It enables practitioners to easily define skeletons and additional hypotheses that sufficiently reduce the search space to mitigate structural ambiguity, facilitating the discovery of interpretable and physically consistent governing equations.

Structural skeletons

The `PyCC` library operates by enforcing a structural skeleton, which serves as a template to define a family of admissible physical models. Unlike purely data-driven 'black-box' methods, this hypothesis-driven approach constrains the model to a physically motivated structure, ensuring that the identified components are structurally identifiable and grounded in the underlying physics.

The library decomposes global dynamics into these skeletons composed of univariate one-dimensional (1D) functions. The following examples are skeletons with structural identifiability properties [6]

When reconstructing governing equations from empirical observations, practitioners face the ill-posed nature of the inverse problem: multiple distinct mathematical models can often yield similar prediction errors, making model selection ambiguous and failing to guarantee a unique representation of the true underlying mechanisms. This identifiability problem is particularly relevant in purely data-driven methods, where the lack of constraints allows algorithms to find spurious models that properly fit the data but fail to capture the actual physics. Existing approaches face significant limitations in addressing this challenge:

Opaque predictive models: Methods like Neural ODEs [7] treat the entire vector field $\mathbf{F}$ as a monolithic NN. Multiple distinct internal representations can produce similar predictions, leading to models that may be flexible and precise but fail to be consistent with the underlying physics [8]. This lack of uniqueness makes physical interpretation difficult or impossible.

Interpretable sparse and symbolic regression methods: Approaches based on sparse regression (e.g., SINDy [9]) and symbolic regression (e.g., PySR [10]) aim to find analytical equations from data. However, these methods often face similar identifiability challenges; depending on the candidate library or set of operations, fundamentally different system structures may yield nearly identical error values. This leads to model non-uniqueness, where multiple non-equivalent equations appear equally valid.

Structured Discovery with Prior Knowledge

One way to address these identifiability issues is by incorporating physical prior knowledge about the system. `PyCC` package addresses this by using the formalism of CCs, which enables practitioners to:

1. **A first-order family:** With applications in overdamped systems such as mechanical nonlinear damping, viscoelastic materials, nonlinear RL series and RC parallel circuits [3, 4].

$$F_{ext}(t) = f_1(x) + f_2(x)\dot{x} \tag{1}$$

2. **A second-order family with velocity-dependent friction:** The standard model for mechanical oscillators with velocity-dependent friction, also known as a generalized Rayleigh-type nonlinear oscillation with velocity-dependent friction and external forcing [5, 6].

$$F_{ext}(t) = \ddot{x} + f_1(\dot{x}) + f_2(x) \tag{2}$$

3. **A second-order family with position-dependent friction:** Designed for systems where damping varies with position, also known as a Liénard equation with external forcing [5].

$$F_{ext}(t) = \ddot{x} + f_1(x)\dot{x} + f_2(x) \tag{3}$$

4. **A coupled multi-degree-of-freedom (MDOF) family:** Modeling the interactions for a coupled system [6, 11].

$$F_{ext,1}(t) = \ddot{x}_1 + f_1(\dot{x}_1) + f_2(x_1 - x_2)$$
$$F_{ext,2}(t) = \ddot{x}_2 + f_3(\dot{x}_2) + f_4(x_2 - x_1)$$

The library enables the user to seamlessly define their own proposed skeletons while also integrating specific physical priors directly into the f_i functions, such as enforcing odd/even parity for f_1, or specifying values like $f_2(1) = 0$.

The f_i functions can be strategically defined to match the constitutive relations or CCs of the system. For instance, in the velocity-dependent friction model (Eq. (2)), f_1 and f_2 are associated with dissipative (friction) and elastic (stiffness) elements, respectively.

Depending on the domain knowledge of the practitioner and the desired level of interpretability, these 1D functions can be identified using three parametrizations:

- **Functional ($\{f_i\}$; $i = 0, 1, \ldots$):** The 1D functions are treated as unknown functions and approximated, for instance, using universal approximators such as Neural Networks (NNs) or Symbolic Regression (SymbR).
- **Parametric ($\{a_i\}$; $i = 0, 1, \ldots$):** The functional form is completely hypothesized and parameterized using scalar parameters $\{a_i\}$. The library identifies the optimal values for $\{a_i\}$ through nonlinear optimization.
- **Hybrid ($\{f_i\}, \{a_j\}$; $i, j = 0, 1, \ldots$):** This approach enables practitioners to simultaneously fit 1D functions $\{f_i\}$ and parameters $\{a_j\}$.

Relation to other packages

`PyCC` can be considered as an integrative framework within the scientific machine learning ecosystem. Its primary differentiator is a hypothesis-driven workflow that shifts the focus from purely data-driven discovery to a structural 'skeleton' approach, ensuring physical consistency and structural identifiability. While many packages seek to find the 'best' equation from scratch, this library provides a formal pipeline to validate if a hypothesized physical structure is consistent with the measured dynamics.

The library is not intended to replace or compete with established packages, but rather to serve as an interface that uses their strengths (such as the symbolic power of `PySR` and the flexibility of NNs) within a structured framework that prioritizes identifiability, interpretability, and physical consistency.

- **Sparse identification (e.g., `SINDy` [9]):** This approach is highly effective when the system dynamics can be represented as a sparse combination of terms from a pre-defined library. However, its success is highly dependent on the user correctly guessing the necessary basis functions. If a complex or non-standard term (such as a specific friction model or a saturation curve) is absent, the results may be misleading. While the CC-based formalism can theoretically be implemented via sparse regression, the `PyCC` library focuses on reconstructing the *shape* of unknown constitutive relations directly. By using, for instance, universal approximators like NNs or SymbR, `PyCC` captures arbitrary functional forms without requiring a rigid prior library of basis terms.
- **Symbolic regression (e.g., `PySR` [10]):** Standard symbolic regression tools often require searching vast spaces to discover relationships in multivariate data. In contrast, `PyCC` acts as a manager that internally uses `PySR` through two specialized workflows:
  - *Iterative symbolic discovery:* This method decomposes the multivariate problem into a sequence of simpler, 1D symbolic regression that iteratively minimize the residual according to the user-defined skeleton.
  - *Symbolic post-processing:* The library features an automated post-processing tool to extract analytical expressions from identified CCs (obtained, for instance, via the NN method). This hybrid strategy has demonstrated superior robustness to noise compared to purely NN-based identification [6].
- **Neural ODEs:** Standard implementations [7] usually learn entire vector fields using a single, monolithic NN. While powerful for prediction, these "black-box" models offer limited physical insight. `PyCC` adopts a 'grey-box' philosophy by modeling only the specific, unknown constitutive relations within a fixed structural skeleton. This ensures that the learned components (such as damping or restoring elements) retain a clear, isolated physical interpretation regardless of the numerical backend used.
- **Physics-informed neural networks (PINNs):** `PINNs` [12] excel at solving forward and inverse problems by embedding physical laws into a loss function. However, they are often less suited for discovery when the functional forms of interactions are entirely unknown. `PyCC` differs by explicitly isolating these unknown terms as distinct functions (f_i) to be learned, making them directly accessible for independent visualization, analysis, and physical validation.

Features

`PyCC` is designed to be a user-friendly and highly-customizable tool for researchers and practitioners. Its key features include:

- **Interpretable models:** It allows practitioners to add prior information by decomposing the complex, high-dimensional functional space into a set of simple, one-dimensional CCs that have direct physical meaning (i.e., the CCs are directly the constitutive relations of the system elements, such as, stiffness or damping), and a set of scalar parameters (e.g., mass).
- **Flexible function parameterization:** Supports multiple back-ends for modeling the 1D functions, allowing practitioners to switch between different representations with minimal modifications:

  - **Neural Networks (NN):** Implemented using `PyTorch` [13] and compatible with `GPUs` and multi-core `CPUs`. Natively supports both NVIDIA ('cuda') and Intel ('xpu' via "intel-extension-for-pytorch") `GPUs` for training the NNs.
  - **Polynomials (Poly):** Provides an expansion of the f_i functions using polynomial basis functions.
  - **Symbolic regression (SymbR):** Utilizes the `PySR` package [10] to discover analytical expressions that are compatible with a given prior structure, while also serving as a post-processing tool.

- **Physics-informed discovery:** Allows users to inject domain knowledge as constraints during training (e.g., 'f1 odd', and 'f2(0)=0') or by defining conserved quantities that are added to the loss function. This leads to more robust and physically consistent models.
- **Hardware acceleration:** Natively supports multicore CPUs and GPUs from both NVIDIA ('cuda') and Intel ('xpu' via 'intel-extension-for-pytorch') for training the NNs.
- **Built-in simulator:** Includes a versatile ODE solver (`pycc.simulate`) compatible with all identification methods. This solver facilitates forward integrations of both the identified models and the theoretical equations used to generate training databases.
- **Comprehensive documentation:** Provides a quick-start Google Colab tutorial with an accompanying YouTube video, along with a complete documentation, examples, and recommended workflows.

Core functions and methods

This section briefly describes the three main methods that are defined in the `PyCC` library: `pycc.simulate()`, `pycc.train()`, and `pycc.post_processing()`.

1. `pycc.simulate()`

This function is responsible for forward system integrations over time. It acts as a simulation manager, dispatching the task to different methods (such as 'Theoretical', 'NN', 'SymbR', 'Poly', or 'Interp'). It has two main uses:

- *Theoretical simulation:* Can be used to integrate forward the theoretical equations in order to obtain ground-truth datasets that will be used for training the models, or to generate theoretical forward integration under different initial conditions or driven forces to compare against model predictions.
- *Validation:* Integrates discovered models after training (e.g., from a NN model) to verify their accuracy against expected dynamics.

2. `pycc.train()`

This function serves as the primary entry point for identifying system dynamics from a given dataset. It operates as a training dispatcher that abstracts the complexity of individual algorithms, automatically routing data to the appropriate module based on the specified "method" parameter (such as 'NN', 'SymbR', 'Poly').

3. `pycc.post_processing()`

This acts as a standalone utility, specifically designed to obtain analytical expressions for the obtained CCs. It converts numerical CCs (previously obtained from methods such as 'NN' or 'Poly') into explicit symbolic expressions. It has two main uses:

- *Model conversion:* Transforms the obtained CCs into symbolic functions for inspection, analysis, and interpretation.
- *Pre-simulation preparation:* Generates optimized symbolic models that are fully compatible with `pycc.simulate()` for subsequent simulations.

Illustrative example: A second-order ODE

To demonstrate the capabilities of `PyCC`, we consider a classic physical system: a nonlinear oscillator with friction. The governing equation is a second-order differential equation:

$$\ddot{x} + \delta\dot{x} + \mu \tanh(500\dot{x}) + \alpha x + \beta x^3 = F_{ext}(t)$$

where $F_{ext}(t) = A\cos(\omega t)$ is an external driving force. The term $\tanh(500\dot{x})$ acts as a smooth approximation of the signum function, $\text{sign}(\dot{x})$, effectively modeling Coulomb friction.

To apply the PyCC framework, we rewrite this system as a set of first-order ordinary differential equations (ODEs). By defining the state variables $x_1 = x$ (position) and $x_2 = \dot{x}$ (velocity), the system becomes:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = F_{ext}(t) - \delta x_2 - \mu \tanh(500 x_2) - \alpha x_1 - \beta x_1^3 \end{cases}$$

The system is simulated with defined parameters and initial conditions to generate the dataset $\mathcal{D} = \{x_1, x_2, \dot{x}_1, \dot{x}_2, F_{ext}\}$, which provides the input for system identification. The following code shows how to use the PyCC library to integrate these theoretical equations in order to generate the input dataset that will be used later for training the models:

```
import pycc
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

################################################
# Simulating a stick-slip second order system using
# pycc.simulate(method='Theoretical')
# Defining parameters and theoretical functions
alpha=1.0;beta=0.2;delta=0.1;Omega=1.0;
x0=0.0;v0=0.0; y0=[x0,v0] # initial conditions
t_span=(0, 20); t_eval=np.linspace(*t_span, 1000)
def F1_th(x_dot):
    return delta * x_dot + 0.5 * np.tanh(500*x_dot)
def F2_th(x):
    return alpha * x + beta * x**3
def F_ext(t):
    return np.cos(Omega * t)
# Defining the theoretical equation
eqs_th = ['x1_dot = x2',
          'x2_dot = F_ext - f1(x2) - f2(x1)']
# Defining the simulation parameters
params_th = {
    't_span': t_span,
    'y0': y0,
    't_eval': t_eval,
    'method': 'LSODA',
    'local_funcs': {'f1': lambda t: F1_th(t),
                    'f2': lambda t: F2_th(t),'F_ext': lambda t: F_ext(t)}
}
# Integrating forward the theoretical equation
sol,derivatives = pycc.simulate(eqs_th,method='Theoretical', params=params_th)
# Extracting data
time_data    = sol.t
x1_data      = sol.y[0]
x2_data      = sol.y[1]
x1_dot_data  = derivatives[0]
x2_dot_data  = derivatives[1]
F_ext_val    = F_ext(time_data)
```

Identification strategies

PyCC supports three distinct identification strategies, categorized by the level of prior knowledge incorporated into the model:

(i) Functional approach In this approach, we assume a *structural skeleton* for the physics but leave the specific constitutive relations as unknown functions to be learned from data. The practitioner hypothesizes that the system consists

of a velocity-dependent damping force and a position-dependent restoring force:

$$\ddot{x} = F_{ext}(t) - f_1(\dot{x}) - f_2(x) \tag{4}$$

This family of second order systems is called *velocity-dependent friction models with external force* and has structural identifiability properties as discussed in Refs. [5, 6]. Translating this to the state-space representation recommended by `PyCC`:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = F_{ext}(t) - f_1(x_2) - f_2(x_1) \end{cases}$$

The goal: Discover the shapes of the CCs $f_1(x_2)$ and $f_2(x_1)$. These functions can be approximated using neural networks (NN), symbolic regression (SymbR), or polynomial expansions (Poly).

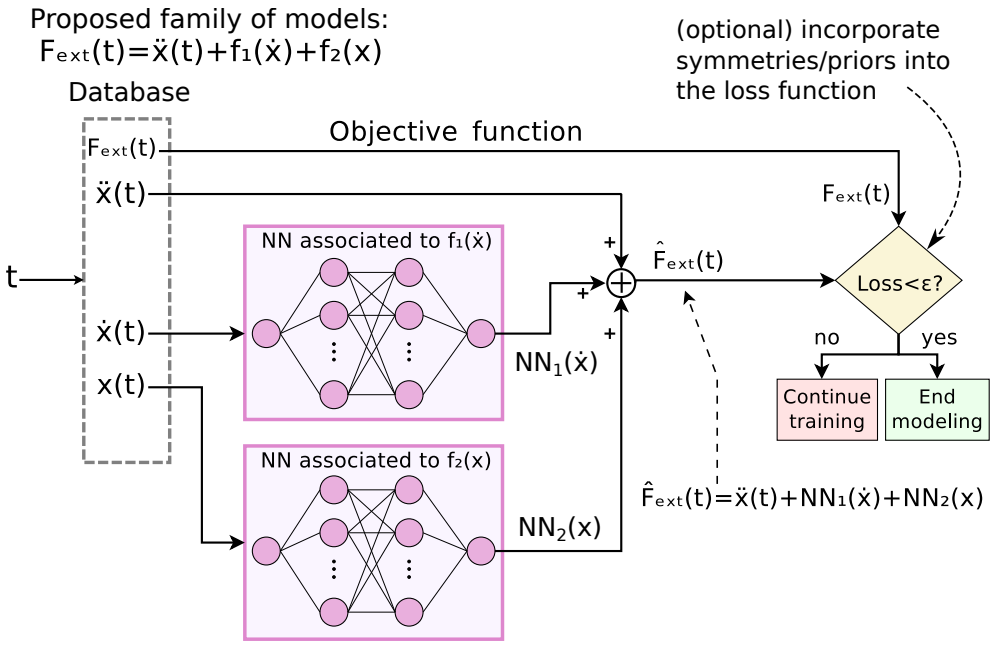

Figure 1: The architecture for a second-order system. Two independent neural networks (NN_1 and NN_2) approximate the unknown CCs. NN_1 sees only velocity, and NN_2 sees only position. Adapted with permissions from Ref. [6].

Figure 1 illustrates the internal architecture automatically generated by `PyCC` when the `method='NN'` is specified as input parameter for the provided equations. The following code snippet shows the required implementation workflow for the user.

```
##############################################
# Defining the database for training
df = pd.DataFrame({
    'x1':x1_data,
    'x2':x2_data,
    'x1_dot':x1_dot_data,
    'x2_dot':x2_dot_data,
    'F_ext': F_ext_val
})
# Defining the proposed equations to use for identification
eqs = [
     'x1_dot = x2',
     'x2_dot = F_ext - f1(x2) - f2(x1)'
]
# In this example, we use the functional approach
# with the NN-CC method [pycc.train(method='NN')]
constraints = [ # adding prior known information
    {'constraint': 'f2(0)=0'},
```

```
    {'constraint': 'f1 odd'},
    {'constraint': 'f2 odd'},
]
# defining training parameters (optional)
params_NN = {
    'neurons': 100,
    'layers':3,
    'lr': 1e-4,
    'epochs': 2000,
    'error_threshold': 1e-6,
    'extrapolation': None,
    'device':'cpu',
    'weight_loss_param': 1e-3,
    'constraints': constraints,
}
# training the model using NN-CC method
models, evals, obtained_coefs = pycc.train(df, eqs,method='NN', params=params_NN)
# plotting obtained functions f1 and f2
x_f1_cc, f1_cc, x_f2_cc, f2_cc = evals
fig, ax = plt.subplots(1, 2, figsize=(12, 6))
ax[0].plot(x_f1_cc, f1_cc, label='$f_1$ learned NN-CC')
ax[0].plot(x_f1_cc, F1_th(x_f1_cc), '--', label='$f_1$ theory')
ax[0].set_xlabel('$x_2$')
ax[0].set_ylabel('$f_1(x_2)$')
ax[0].legend()
ax[1].plot(x_f2_cc, f2_cc, label='$f_2$ learned NN-CC')
ax[1].plot(x_f2_cc, F2_th(x_f2_cc), '--', label='$f_2$ theory')
ax[1].set_xlabel('$x_1$')
ax[1].set_ylabel('$f_2(x_1)$')
ax[1].legend()
plt.tight_layout()
plt.show()

# Print learned parameters (if any)
if obtained_coefs:
    print('\nLearned scalar parameters:')
    for name, val in obtained_coefs.items():
        print(f'{name} = {val.item():.4f}')

################################################
# simulating forward the identified model
# using NN-CC method [pycc.simulate(method='NN')]
params_NN_simul = {
    'models': models,
    'obtained_coefs': obtained_coefs,
    'local_funcs': {'F_ext': lambda t: F_ext(t)},
    't_span':t_span,
    'y0': y0,
    't_eval': t_eval,
    'method': 'LSODA',
    'atol': 1e-8,
    'rtol': 1e-6,
    'check_nan': True
}
# integrating identified equations
sol,_ = pycc.simulate(eqs, method='NN', params=params_NN_simul)
time_sim=sol.t
x1_sim=sol.y[0]
x2_sim=sol.y[1]
```

```
# Plotting identified vs theoretical solution
plt.figure()
plt.plot(time_sim, x1_sim, label='x(t) simulated with NN method')
plt.plot(time_data, x1_data, label='x(t) th')
plt.xlabel('t')
plt.ylabel('x(t)')
plt.legend()
plt.show()
```

Additionally, the following script demonstrates how to use the 'evals' variable to perform a post-processing workflow in which the previously obtained CCs are fitted analytically and then simulated forward using, in this example, a `pchip` interpolation.

```
################################################
# Post-processing the identified CCs with
#       SymbR [pycc.post_processing()].
print('post-SR processing for the CCs')
# Defining settings for the PySR fit
pysr_settings = {
    'niterations': 100,
    'populations': 20,
    'binary_operators': ['+', '*', '-'],
    'unary_operators': ['tanh', 'sin','cos'],
    'maxsize': 20
}
# Defining the 'params' dictionary for the post-processing function
post_process_params = {
    'evals': evals,
    'pysr': pysr_settings,
    'plot': True,
    'n_eval': 200,
}
# Running the post-processing
models_sr,evals_sr = pycc.post_processing(eqs, method='SymbR', params=post_process_params)

################################################
# We can use the post-processed CCs for forward simulation
params_sim_interp = {
    'evals': evals_sr,
    'obtained_coefs': obtained_coefs,
    'local_funcs': {'F_ext': F_ext},
    'interp_method': 'pchip',
    't_span': t_span,
    'y0': y0,
    't_eval': t_eval,
}
# We simulate the system by interpolating the CCs
sol, derivs = pycc.simulate(eqs, method='Interp', params=params_sim_interp)
```

(ii) Parametric Approach

In this approach, the practitioner explicitly defines the symbolic structure of the candidate functions using parameters $\{a_i\}$, which `PyCC` then optimizes to fit the data:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = F_{ext}(t) - a_1\, x_2 - a_2 \tanh(a_3\, x_2) - a_4\, x_1 - a_5\, x_1^3 \end{cases}$$

The goal: Identify the optimal scalar parameters $\{a_i\}$ that minimize the error between the model and the data.

```
# Defining the proposed equations to use for identification
eqs = [
     'x1_dot = x2',
     'x2_dot = F_ext - a1 x2 - a2 tanh(a3 x2) - a4 x1 - a5 x1^3'
]
```

(iii) Hybrid Approach

PyCC also supports a hybrid approach that integrates functional discovery with parametric optimization. For example:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = F_{ext}(t) - f_1(x_2) - a_4\, x_1 - a_5\, x_1^3 \end{cases}$$

```
# Defining the proposed equations to use for identification
eqs = [
     'x1_dot = x2',
     'x2_dot = F_ext - f1(x2) - a4 x1 - a5 x1^3'
]
```

PyCC reserves the names f_i and a_i to represent functions and scalar parameters, respectively.

Using other methods

Alternative methods to NN such as Poly and SymbR can be used with only minor code changes. For instance, the following code shows the implementation of the SymbR method:

```
params_SymbR = {
  'pysr': {
    'niterations': 100,
    'unary_operators': ['sin','cos','tanh'],
    'binary_operators': ['+','-','*'],
    'maxsize': 12,
    'populations':10,
    'model_selection': 'accuracy',
    'verbosity': 0
  },
  'N_fit_points': 200,
  'max_iterations': 25,
}
models, evals , obtained_coefs = pycc.train(
    df=df,
    equations=equations,
    method='SymbR',
    params=params_SymbR
)

sol,_ = pycc.simulate(equations, method='SymbR', params=params_SR_simul)
```

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The software package is available at: https://github.com/FedejGon/pyCC.id

AI usage disclose

The core structure of the source code was authored manually. AI tools were subsequently employed during the development phase to assist with code refactoring and the generation of comments in the code. All AI-assisted components were rigorously reviewed, tested, and validated by the author to ensure functional integrity.

Additionally, AI tools were utilized during the drafting process of this manuscript to provide feedback on clarity and wording. All AI-generated suggestions were subject to critical revision and final approval by the author.

Acknowledgments

The author acknowledges fruitful discussions with Luis P. Lara, Carlos E. Repetto, Bernardo J. Gómez, Rodolfo Id Betán, Ignacio Pomponio, and Luis Manuel. This work was supported by ANPCyT Project PICT-2021-I-A-01135, CONICET Project PIP 1679, and the UNR Project PID 80020190100011UR (Argentina). The author acknowledges the CCT-Rosario Computational Center for the provision of computing resources and the Secretaría de Innovación, Ciencia y Tecnología (SICYT) of Argentina for access to Clementina XXI supercomputer (project PCI-91), both of which were used to develop and test this library.

References

[1] Felix Dietrich and Wil Schilders. "Scientific machine learning". In: *Mathematische Semesterberichte* 72.2 (2025), pp. 89–115. ISSN: 1432-1815. DOI: 10.1007/s00591-025-00399-4.

[2] Carola-Bibiane Schönlieb and Zakhar Shumaylov. *Data-driven approaches to inverse problems*. 2025. arXiv: 2506.11732 [math.NA]. URL: https://arxiv.org/abs/2506.11732.

[3] F. J. Gonzalez. "Determination of the characteristic curves of a nonlinear first order system from Fourier analysis". In: *Sci. Rep.* 13.1 (Feb. 2023), p. 1955. DOI: 10.1038/s41598-023-29151-5.

[4] F. J. Gonzalez. "System identification based on characteristic curves: a mathematical connection between power series and Fourier analysis for first-order nonlinear systems". In: *Nonlinear Dyn.* 112.18 (July 2024), pp. 16167–16197. ISSN: 1573-269X. DOI: 10.1007/s11071-024-09890-4.

[5] F. J. Gonzalez and L. P. Lara. "Interpretable neural network system identification method for two families of second-order systems based on characteristic curves". In: *Nonlinear Dyn.* 113.24 (Sept. 2025), pp. 33063–33086. ISSN: 1573-269X. DOI: 10.1007/s11071-025-11744-6.

[6] F. J. Gonzalez. "Integrating prior knowledge in equation discovery: Interpretable symmetry-informed neural networks and symbolic regression via characteristic curves". In: (2026). arXiv: 2601.21720 [nlin.CD]. URL: https://arxiv.org/abs/2601.21720.

[7] R.T.Q. Chen et al. "Neural Ordinary Differential Equations". In: *NeurIPS 2018*. Ed. by S. Bengio et al. Vol. 31. Curran Associates, Inc., 2018.

[8] K. Wu. "Automations Black-Box Conundrum: The Interpretability Crisis of Data-Driven System Identification and the Path Forward". In: *Theoretical and Natural Science* 120.1 (July 2025), pp. 25–35. ISSN: 2753-8826. DOI: 10.54254/2753-8818/2025.ad25063. URL: http://dx.doi.org/10.54254/2753-8818/2025.AD25063.

[9] S. L. Brunton, J. L. Proctor, and J. N. Kutz. "Discovering governing equations from data by sparse identification of nonlinear dynamical systems". In: *Proceedings of the National Academy of Sciences* 113.15 (2016), pp. 3932–3937. DOI: 10.1073/pnas.1517384113.

[10] M. Cranmer. "Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl". In: (2023). arXiv: 2305.01582. URL: https://arxiv.org/abs/2305.01582.

[11] Ali H. Nayfeh and P. Frank Pai. *Linear and Nonlinear Structural Mechanics*. New York, NY: John Wiley & Sons, Aug. 2004. DOI: 10.1002/9783527617562.

[12] M. Raissi, P. Perdikaris, and G.E. Karniadakis. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations". In: *Journal of Computational Physics* 378 (2019), pp. 686–707. ISSN: 0021-9991. DOI: 10.1016/j.jcp.2018.10.045.

[13] Adam Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *NeurIPS 32*. Ed. by H. Wallach et al. Curran Associates, Inc., 2019, pp. 8024–8035.